



Université Sultan Moulay
Slimane

École Nationale des Sciences
Appliquées
-Khouribga-



RAPPORT DE PROJET

Design, Implementation, and Deployment of a Client–Server Speech-to-Text System

Module : Technology JS

Filière Informatique et Ingénierie des Données

Realised by :

Achraf ZAHID

Mohamed Amine EL ARBANI

Salma IDRISSE LAMGHARI

Assiya KOURAA

Supervised by :

Pr. Amal OURDOU

Année Universitaire 2025-2026

Acknowledgements

We wish to express our sincere gratitude to all the people who contributed, directly or indirectly, to the completion of this project.

Our warmest thanks go first and foremost to our supervisor **Pr. Amal OURDOU**, for her availability, invaluable advice, and continuous support throughout this work. Her guidance and expertise were decisive in orienting our technical and methodological choices.

We also thank the **teaching staff** of the École Nationale des Sciences Appliquées de Khouribga, and in particular the lecturers of the Computer Science and Data Engineering programme, for the quality of the education provided and the skills acquired throughout our studies.

Abstract

Moroccan Darija, a predominantly spoken Arabic dialect with limited written standardization, remains severely underserved by commercial automatic speech recognition (ASR) systems. This report presents [Sowelni](#), an end-to-end, real-time voice assistant designed specifically for Darija speakers. The system captures microphone audio in the browser, streams it over WebSockets to a Python backend, transcribes it using a fine-tuned Qwen3-ASR-0.6B model, and generates intelligent conversational responses via a large language model (LLM).

The architecture employs Angular 19 with reactive Signals for the frontend, FastAPI with `python-socketio` for the backend, and the official `qwen-asr` Python package for non-blocking GPU inference. Audio is captured using the browser's native `MediaRecorder` API with a stop-restart chunking strategy that produces standalone WebM blobs every two seconds, each independently decodable by the backend. LLM responses are streamed token-by-token back to the UI, providing an interactive conversational experience.

We document the complete system design, the fine-tuning methodology for adapting Qwen3-ASR to Darija (including a detailed analysis of a failed training run and the corrective measures applied), the real-time audio pipeline, the WebSocket protocol, and deployment considerations. The project is fully open-source and designed to run entirely on local hardware, with the LLM component being the only optionally external dependency.

Keywords: Automatic speech recognition, Moroccan Darija, low-resource language, Qwen3-ASR, fine-tuning, real-time streaming, Angular 19, FastAPI, WebSockets, LLM integration.

Contents

Acknowledgements	2
Abstract	3
List of Figures and Tables	6
General Introduction	7
1 Context and Problem Statement	8
1.1 General Context	8
1.1.1 The ASR Landscape	8
1.1.2 Real-Time Speech Applications	9
1.2 Specific Context: Moroccan Darija	9
1.3 Problem Statement	10
1.4 Project Objectives	10
2 Methodology	12
2.1 Overall System Architecture	12
2.1.1 High-Level Overview	12
2.1.2 Data Flow	13
2.1.3 Design Principles	14
2.2 Backend Implementation	14
2.2.1 Technology Stack	14
2.2.2 Application Entry Point	15
2.2.3 ASR Service	15
2.2.4 LLM Service	15
2.2.5 Session Management	16
2.2.6 Audio Processing	16

2.3	Frontend Implementation	16
2.3.1	Angular 19 and Signals	16
2.3.2	Angular 19 Control Flow Syntax	17
2.3.3	WebSocket Service	17
2.3.4	Live Waveform Visualisation	17
2.4	Real-Time Audio Pipeline	18
2.4.1	The WebM Chunking Problem	18
2.4.2	The Stop-Restart Solution	18
2.4.3	Latency Budget	19
2.5	WebSocket Protocol	19
2.6	User Interface Design	20
3	Experimentation and Results	22
3.1	Experimental Environment	22
3.1.1	Hardware Configuration	22
3.1.2	Software Stack	23
3.2	ASR Model Fine-Tuning	23
3.2.1	Base Model	23
3.2.2	Training Data	23
3.2.3	Failed Training Run: Diagnosis	23
3.2.4	Corrected Training Configuration	24
3.2.5	Expected Results	25
3.3	Frontend Screenshots	25
3.4	Deployment and Operations	27
3.4.1	Startup Sequence	27
3.4.2	Security Considerations	27
	Conclusion	28
	References	30

List of Figures

Fig. 2.1 – High-level system architecture: data flows from the browser (left) through the server pipeline (right), with transcription and LLM tokens streamed back to the UI (dashed arrows).	13
Fig. 3.1 – Idle state: the application is connected and waiting for the user to press the microphone button.	26
Fig. 3.2 – Recording state: the microphone button turns red and the 24-bar waveform visualisation animates around it, providing live audio level feedback.	26
Fig. 3.3 – Transcription in progress: the Darija speech is transcribed in real time and displayed in the user bubble as each 2-second chunk is processed.	27

List of Tables

Tab. 2.1 – Backend technology stack.	15
Tab. 2.2 – Reactive signals and their roles in the UI.	17
Tab. 2.3 – Latency breakdown for transcription (not including LLM).	19
Tab. 2.4 – Complete WebSocket event protocol.	20
Tab. 3.1 – Hardware requirements for the backend.	22
Tab. 3.2 – Software prerequisites.	23
Tab. 3.3 – Metrics from the failed training run.	24
Tab. 3.4 – Corrected training hyperparameters.	24
Tab. 3.5 – Expected WER/CER ranges after corrected fine-tuning.	25

General Introduction

Automatic Speech Recognition (ASR) converts an audio signal into text and plays a central role in voice assistants and human-machine interfaces. Recent models have considerably improved ASR performance, but primarily for well-resourced languages. Moroccan Darija, spoken by more than 30 million people, remains marginalised: it lacks orthographic standardisation, features frequent code-switching with French, English, and Amazigh, and exhibits significant regional phonetic variation. This shortage of resources hinders the development of voice applications for Moroccan speakers.

The [Sowelni](#) project addresses this gap through the following objectives: (1) build a complete, real-time browser-based voice assistant specifically for Darija speakers; (2) fine-tune a state-of-the-art multilingual ASR model (Qwen3-ASR-0.6B) on 40 hours of Darija speech data; (3) design a local-first architecture that runs the ASR pipeline entirely on a consumer GPU; and (4) achieve perceived end-to-end latency under 5 seconds through streaming at every stage of the pipeline.

This report is organised into three chapters: Chapter 1 presents the theoretical context (ASR fundamentals, Darija specificities, related work); Chapter 2 details the methodology (system architecture, backend, frontend, audio pipeline, and WebSocket protocol); Chapter 3 presents the ASR fine-tuning experiments, results, frontend screenshots, and deployment considerations.

Chapter 1

Context and Problem Statement

This chapter presents the theoretical and contextual foundations of the Sowelni project. We first survey the current ASR landscape and the role of real-time speech applications, before examining the specific linguistic characteristics of Moroccan Darija that make it a particularly challenging target for automatic recognition. We then formulate the research question that motivates the project and define the concrete objectives pursued throughout this work.

1.1 General Context

Automatic Speech Recognition (ASR) has undergone a profound transformation in recent years, driven by end-to-end neural architectures and the availability of large-scale datasets. While these advances have achieved remarkable performance on major languages, they mask a far more contrasted reality: a large portion of the world’s languages and dialects remains severely underrepresented, or entirely absent, from training corpora. Moroccan Darija perfectly illustrates this digital divide.

1.1.1 The ASR Landscape

The landscape of open-source ASR has been transformed by large-scale multilingual models. OpenAI’s Whisper [6] demonstrated that a single encoder–decoder Transformer [10] trained on 680,000 hours of weakly supervised data could achieve competitive performance

across 97 languages. However, Whisper’s Arabic performance is dominated by Modern Standard Arabic (MSA), and its Darija accuracy remains poor without fine-tuning.

The Qwen team’s Qwen3-ASR family [5], built on the Qwen3-Omni foundation model [4], supports 52 languages and dialects. The 0.6B parameter variant offers an attractive balance between accuracy and inference cost, making it suitable for deployment on consumer GPUs. Crucially, the Qwen3-ASR architecture is designed for fine-tuning on new languages and dialects, making it an ideal base for Darija adaptation.

1.1.2 Real-Time Speech Applications

Browser-based real-time speech applications face a fundamental tension: deep learning inference requires GPU hardware that the browser does not expose, necessitating a client–server architecture with streaming transport. The Web Speech API provides basic speech recognition but offers no control over the underlying model and does not support custom languages or dialects.

WebSocket-based architectures have emerged as the standard approach. Socket.IO [9] provides a reliable abstraction over WebSockets with automatic reconnection, binary frame support, and event-based messaging. Combined with a Python backend, this enables a clean separation between the browser-based audio capture and the GPU-based inference.

1.2 Specific Context: Moroccan Darija

Moroccan Darija is the mother tongue of approximately 30–40 million speakers across Morocco and the Moroccan diaspora. Unlike Modern Standard Arabic (MSA), Darija is primarily a spoken language characterised by heavy borrowings from Amazigh, French, and Spanish, non-standardised orthography, and significant regional variation. These properties make it exceptionally challenging for ASR systems trained predominantly on MSA or other well-resourced languages.

Commercial ASR services from major providers (Google Speech-to-Text, Amazon Transcribe, Microsoft Azure Speech) offer “Arabic” as a language option, but their models are overwhelmingly trained on MSA and Gulf Arabic, resulting in poor accuracy on Darija. This gap creates a tangible barrier: Darija speakers cannot reliably use voice assistants,

dictation tools, or accessibility software in their native dialect.

Previous approaches to Darija ASR have relied on either (a) training from scratch on small corpora, yielding high word error rates (WER), or (b) fine-tuning multilingual models like Whisper, which requires careful hyperparameter selection to avoid catastrophic forgetting. Our approach follows the latter strategy but uses Qwen3-ASR-0.6B as the base model, which benefits from a more modern architecture and stronger pre-training on diverse audio data. We employ LoRA-compatible full fine-tuning [2] with careful label masking to preserve the model’s general audio understanding while specialising it for Darija.

1.3 Problem Statement

Research Question

To what extent is it possible to build a real-time, end-to-end voice assistant for Moroccan Darija that runs on a single consumer GPU, by combining a fine-tuned Qwen3-ASR-0.6B model with a streaming Angular 19 / FastAPI architecture?

1.4 Project Objectives

The [Sowelni](#) project addresses the above question with the following concrete objectives:

1. **Real-time Darija ASR:** Fine-tune Qwen3-ASR-0.6B on 40 hours of Darija speech data to achieve usable word error rates on conversational Darija.
2. **End-to-end voice assistant:** Build a complete client–server application where users speak Darija into their browser, see a live transcription, and receive an intelligent response from an LLM.
3. **Local-first architecture:** Ensure that the ASR pipeline runs entirely on local hardware (a single consumer GPU), with the LLM being the only component that optionally depends on an external API.
4. **Low-latency interaction:** Achieve perceived end-to-end latency under 5 seconds (from end-of-speech to first LLM token appearing in the UI) through streaming at every stage.

This chapter has highlighted the dual nature of the challenge that Moroccan Darija poses for ASR systems: a living language spoken by millions, yet lacking orthographic standardisation and almost entirely absent from mainstream training corpora. The detailed methodology for addressing this challenge is presented in the following chapter.

Chapter 2

Methodology

This chapter describes all the methodological choices adopted to design the [Sowelni](#) system. We first present the overall system architecture, then detail each component: the backend, the frontend, the real-time audio pipeline, and the WebSocket communication protocol.

2.1 Overall System Architecture

2.1.1 High-Level Overview

Sowelni follows a three-tier architecture: a browser-based frontend for audio capture and UI rendering, a Python backend for ASR inference and LLM orchestration, and external services (HuggingFace Hub for model weights, optionally Google Gemini for LLM responses).

The system operates as a continuous pipeline illustrated in Figure 2.1. On the client side, the browser captures the user’s voice through the `MediaRecorder` API and forwards the resulting audio blobs to the server via a `Socket.IO` connection. On the server side, a `FastAPI`/`ASGI` backend receives the binary audio frames, decodes them from `WebM`/`Opus` format into raw `PCM` samples, and routes them through two sequential inference stages: the fine-tuned `Qwen3-ASR-0.6B` model first converts the audio into `Darija` text, which is then forwarded to the LLM (Google Gemini or a local alternative) to generate a conver-

sational response. Both the transcription and the LLM tokens are streamed back to the Angular 19 frontend as they are produced, ensuring that the user sees results progressively rather than waiting for the complete pipeline to finish.

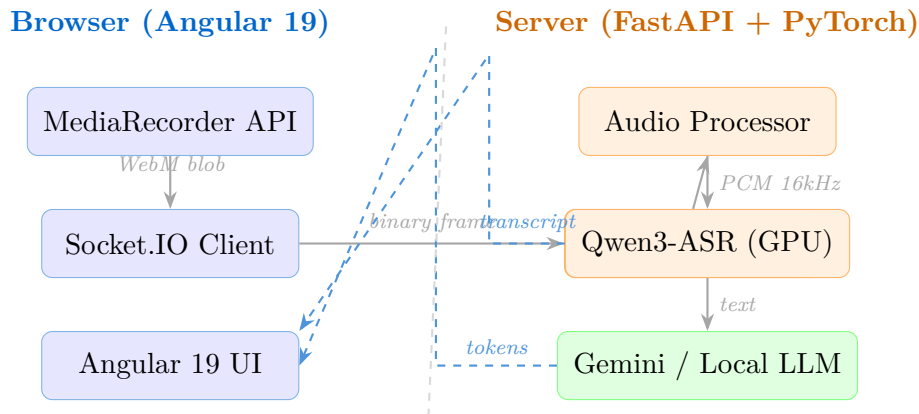


Figure 2.1: High-level system architecture: data flows from the browser (left) through the server pipeline (right), with transcription and LLM tokens streamed back to the UI (dashed arrows).

2.1.2 Data Flow

The complete data flow for a single interaction unfolds in ten stages. First, the Angular frontend requests microphone access via `navigator.mediaDevices.getUserMedia()`, capturing audio as mono with echo cancellation and noise suppression enabled at the browser level. A **MediaRecorder** instance then records for exactly 2 seconds before stopping, producing a complete, standalone WebM/Opus blob with proper EBML headers. This blob's **ArrayBuffer** is immediately emitted over Socket.IO as a binary frame to the backend.

On the server side, **pydub** (backed by **FFmpeg**) decodes the incoming WebM blob into a 16 kHz mono float32 numpy array. Before invoking the ASR model, an RMS-based silence gate discards any chunk that contains no meaningful audio, preventing hallucinations on background noise. The audio array is then passed to the fine-tuned Qwen3-ASR-0.6B model via `model.transcribe()`, running in a dedicated thread to avoid blocking the Socket.IO event loop. The resulting text segment is appended to the session's running transcript and immediately emitted back to the frontend.

Once the user stops recording, the accumulated transcript is forwarded to the LLM (Gem-

ini or a local alternative), which begins generating a conversational response. LLM tokens are streamed back to the frontend as they are produced, and Angular’s reactive Signal system updates the DOM progressively as each transcription segment and token arrives, giving the user immediate visual feedback throughout the interaction.

2.1.3 Design Principles

The architecture is guided by four principles.

Non-blocking I/O. The Socket.IO event loop must remain responsive even during multi-second GPU inference. All blocking operations (model inference, LLM API calls) are dispatched to thread pools and synchronised with an `asyncio` lock.

Graceful degradation. If the LLM is unavailable (no API key, quota exceeded), the ASR pipeline still works. If a single audio chunk fails to decode, the server logs a warning and processes the next one.

Streaming everywhere. No stage waits for the complete result before forwarding. Transcription is sent per-chunk; LLM tokens are forwarded as they arrive; the UI updates reactively.

Local-first. The ASR model and audio processing run entirely on local hardware. The LLM is the only external dependency, and it can be replaced with a local model.

2.2 Backend Implementation

2.2.1 Technology Stack

The backend is built on the following stack:

Component	Technology	Role
Web framework	FastAPI [7]	HTTP endpoints, CORS, health checks
Real-time transport	python-socketio [9]	WebSocket event handling
ASR runtime	qwen-asr (PyPI)	Qwen3-ASR model loading and inference
Audio processing	pydub / FFmpeg [8]	WebM $\rightarrow$ PCM conversion
LLM client	google-genai	Gemini API streaming
Async runtime	uvicorn (ASGI)	Production server

Table 2.1: Backend technology stack.

2.2.2 Application Entry Point

The application uses a composite ASGI stack: `Socket.IO` owns the root path for WebSocket connections, while `FastAPI` handles HTTP requests as a sub-application via `socketio.ASGIApp`. The `max_http_buffer_size` is set to 10 MB, providing sufficient headroom for audio chunks (typical 2-second WebM blobs are 5–30 KB).

2.2.3 ASR Service

The ASR service wraps the official `qwen-asr` package’s `Qwen3ASRModel`. A `ThreadPoolExecutor` with a single worker ensures that only one `model.transcribe()` call runs at any time, while an `asyncio.Lock` prevents multiple concurrent clients from queuing up GPU work. The `Qwen3ASRModel.transcribe()` method accepts audio as a list of numpy array and sample rate tuples; for Darija, the `language` parameter is set to `"Arabic"`, the closest match in the model’s 52-language vocabulary.

2.2.4 LLM Service

The LLM service uses Google’s `google-genai` SDK with streaming. Because the SDK’s streaming iterator is blocking while the `Socket.IO` event loop is asynchronous, a producer–consumer pattern with an `asyncio.Queue` bridges the two: the producer runs in a background thread, pushing tokens into the queue, while the async generator yields them to the `Socket.IO` event handler, which emits each token to the client.

2.2.5 Session Management

Per-connection state is maintained in a dictionary keyed by Socket.IO session ID. Each session tracks the list of transcribed text segments for the current recording and the conversation history (past user questions and model answers) sent to the LLM for multi-turn context. Sessions are created on connect and destroyed on disconnect.

2.2.6 Audio Processing

The audio processor converts incoming WebM/Opus byte streams into the 16 kHz mono float32 arrays expected by the ASR model. The WebM container is decoded via `pydub` (backed by FFmpeg), and the resulting PCM samples are normalised to float32 in $[-1, 1]$. A silence gate based on root-mean-square amplitude then discards near-silent chunks:

$$\text{RMS}(x) = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (2.1)$$

Chunks with $\text{RMS} < 0.01$ are considered silent and discarded, preventing the ASR model from hallucinating text on background noise.

2.3 Frontend Implementation

2.3.1 Angular 19 and Signals

The frontend is built with Angular 19 [1], using the standalone component architecture and the reactive Signals API. Signals provide a synchronous, pull-based reactivity model that is more efficient than RxJS Observables for UI state that updates frequently, such as live transcription text and streaming LLM tokens. Table 2.2 summarises the key signals exposed by the services.

Signal	Description
<code>connected</code>	Boolean. Whether the Socket.IO connection is active.
<code>currentTranscript</code>	String. Running concatenation of all transcribed segments.
<code>currentAnswer</code>	String. Accumulated LLM tokens for the current response.
<code>isStreamingAnswer</code>	Boolean. True while LLM tokens are still arriving.
<code>turns</code>	Array. Completed conversation exchanges (question + answer pairs).
<code>recording</code>	Boolean. Whether the microphone is actively capturing.
<code>levels</code>	Number array (24 elements). Frequency-domain amplitude levels.

Table 2.2: Reactive signals and their roles in the UI.

2.3.2 Angular 19 Control Flow Syntax

Angular 19 introduces a new built-in control flow syntax that replaces structural directives such as `*ngIf` and `*ngFor`. This syntax is compiled at build time rather than evaluated at runtime, resulting in smaller bundle sizes and faster change detection. Conditional rendering uses `@if/@else` blocks, while list rendering uses `@for` with a mandatory `track` expression to enable efficient DOM diffing.

2.3.3 WebSocket Service

The WebSocket service manages the Socket.IO connection lifecycle and translates server events into Signal updates. The socket instance is assigned to a local variable during initialisation before all event listeners are registered, avoiding TypeScript strict-mode issues with nullable fields. Key events handled include `connect`, `transcription`, `llm_token`, `llm_done`, and `error`.

2.3.4 Live Waveform Visualisation

A Web Audio API [11] `AnalyserNode` provides real-time frequency data for the circular waveform visualisation around the microphone button. The analysis loop runs outside Angular's change detection zone via `NgZone.runOutsideAngular()` to prevent unneces-

sary rendering cycles. It downsamples the frequency bins into 24 normalised amplitude values and pushes them into the `levels` signal at each animation frame, driving the visual bars in the template.

2.4 Real-Time Audio Pipeline

2.4.1 The WebM Chunking Problem

A naive approach to audio streaming passes a `timeslice` argument to `MediaRecorder.start()` to slice the stream into segments via the `ondataavailable` callback [3]. However, this produces chunks where only the *first* contains the EBML (Extensible Binary Meta Language) header required for WebM container parsing. Subsequent chunks are raw Opus frames without container metadata, and FFmpeg cannot decode them independently.

2.4.2 The Stop-Restart Solution

Sowelni solves this by stopping and restarting the `MediaRecorder` every 2 seconds. Each recording cycle creates a fresh `MediaRecorder` instance, collects its blobs into a local array, and on the `onstop` event assembles them into a complete, standalone WebM file that is immediately sent to the backend. The gap between segments (the time to stop one recorder and start the next) is approximately 20–40 ms, which is imperceptible in a conversational context.

2.4.3 Latency Budget

Stage	Typical Latency
Audio capture (buffering)	0–2000 ms (chunk duration)
WebSocket transport (LAN)	1–5 ms
WebM decoding (FFmpeg)	20–50 ms
ASR inference (GPU, 2s audio)	200–800 ms
Socket.IO emit (return trip)	1–5 ms
Angular signal update + render	<1 ms
Total (worst case)	≈2.9 s
Average perceived	≈1.5 s

Table 2.3: Latency breakdown for transcription (not including LLM).

The dominant contributor is the 2-second buffering window. This could be reduced by using shorter chunks via an AudioWorklet-based PCM capture approach, at the cost of increased network overhead and more frequent GPU dispatches.

2.5 WebSocket Protocol

The client and server communicate over Socket.IO with the event protocol described in Table 2.4. Audio chunks are sent as raw binary frames rather than base64-encoded strings, reducing bandwidth by approximately 33% and avoiding the CPU cost of encoding on both ends.

Direction	Event	Payload
→ server	audio_chunk	Binary <code>ArrayBuffer</code> (one complete WebM file)
→ server	request_answer	Empty. Triggers LLM call on accumulated transcript.
→ server	reset	Empty. Clears session state and conversation history.
← client	ready	{sid: string}
← client	transcription	{partial: string, full: string}
← client	llm_start	{question: string}
← client	llm_token	{token: string} (emitted per generated token)
← client	llm_done	{answer: string, question: string}
← client	error	{message: string}
← client	reset_ok	Empty.

Table 2.4: Complete WebSocket event protocol.

After each `llm_done` event, the server appends the user’s question and the model’s answer to the session history. This history is passed to the Gemini API on subsequent calls, enabling coherent multi-turn conversation across multiple recording sessions.

2.6 User Interface Design

The **Sowelni** interface deliberately avoids the common dark blue with purple gradients aesthetic prevalent in AI applications. Instead, it uses a warm palette inspired by Moroccan visual culture: a deep warm brown background (`#1a1310`) evoking aged leather and traditional architecture, saffron-amber accents (`#e8a838`) inspired by Moroccan spice markets, and typography combining Cormorant Garamond for the masthead, Amiri for Arabic text, and Manrope for UI elements.

The conversation is displayed as a scrollable series of bubble pairs: user speech (right-aligned, warm tint) and assistant responses (left-aligned, neutral). The centerpiece is a circular microphone button with a gradient fill (saffron when idle, red when recording), a pulsing glow animation during recording, and a 24-bar circular waveform visualisation driven by the Web Audio `AnalyserNode`. The interface is fully responsive down to 320 px

viewport width.

This chapter has presented a rigorous and modular methodology, covering the complete design of the Sowelni system: from the three-tier architecture to the real-time audio pipeline, backend services, reactive frontend, and WebSocket protocol. The ASR fine-tuning experiments, frontend screenshots, and deployment results are presented in the following chapter.

Chapter 3

Experimentation and Results

This chapter presents the results obtained from the Sowelni implementation. We first describe the hardware and software environment, then the ASR fine-tuning methodology and outcomes including a detailed failure analysis, followed by a visual presentation of the deployed frontend and the deployment considerations.

3.1 Experimental Environment

3.1.1 Hardware Configuration

Component	Minimum	Recommended
GPU	4 GB VRAM (e.g., RTX 3060)	8 GB+ (e.g., RTX 4070)
RAM	8 GB	16 GB
CPU	4 cores	8 cores
Storage	5 GB (model + deps)	10 GB

Table 3.1: Hardware requirements for the backend.

The Qwen3-ASR-0.6B model requires approximately 1.2 GB of VRAM in bf16 precision. With CUDA context overhead, practical VRAM usage is approximately 2–3 GB during inference.

3.1.2 Software Stack

Requirement	Version / Notes
Python	3.10+
FFmpeg	Required for pydub WebM decoding
Node.js + Angular CLI	18+ / 19
CUDA + cuDNN	11.8+ / 8+

Table 3.2: Software prerequisites.

3.2 ASR Model Fine-Tuning

3.2.1 Base Model

Qwen3-ASR-0.6B [5, 4] is a 600-million parameter encoder–decoder speech model supporting 52 languages. It processes 16 kHz mono audio through a convolutional feature extractor, followed by a Transformer-based encoder and an autoregressive Transformer decoder that generates text tokens.

3.2.2 Training Data

The fine-tuning dataset consists of 40 hours of Moroccan Darija speech organised as JSONL files, one row per utterance, each containing the path to a mono WAV file and its reference transcription in Arabic script. The dataset is split into training (90%) and validation (10%) sets.

3.2.3 Failed Training Run: Diagnosis

The initial training run produced the following pathological metrics after 16,000 steps:

Metric	Observed Value
Final training loss	4.3–8.3 (oscillating)
Gradient norm (typical)	135–388
Gradient norm (peak)	1,002.8
<code>best_metric</code>	None (no evaluation ran)
Learning rate at step 16,000	8.9×10^{-9}

Table 3.3: Metrics from the failed training run.

Root cause analysis identified four contributing factors. First and most critically, the training loss was computed over *all* tokens including the system prompt and audio placeholder tokens, which are not predictable from the audio input. Second, the absence of gradient clipping allowed gradient norms exceeding 1,000, causing catastrophically large weight updates. Third, without periodic evaluation there was no way to identify the best checkpoint or detect overfitting. Fourth, mixed-precision training with fp16 rather than bf16 introduced numerical instability on Ampere GPUs.

3.2.4 Corrected Training Configuration

Parameter	Value and Rationale
Label masking	Prompt tokens masked with -100. Only transcription tokens contribute to the cross-entropy loss.
<code>max_grad_norm</code>	1.0. Clips gradient norms to prevent explosive updates.
<code>learning_rate</code>	2×10^{-5} (full FT) or 1×10^{-4} (LoRA).
<code>lr_scheduler_type</code>	Cosine with warmup (5% of total steps).
<code>bf16</code>	True on supported hardware (Ampere+).
<code>eval_strategy</code>	Every 500 steps.
<code>load_best_model_at_end</code>	True, metric: <code>eval_loss</code> .
<code>gradient_checkpointing</code>	True. Reduces GPU memory by recomputing activations.

Table 3.4: Corrected training hyperparameters.

The label masking collator is the most critical component: it constructs the full conversation prompt, tokenises it, then sets all prompt token positions to `-100` in the labels tensor, the standard PyTorch sentinel for positions excluded from the loss computation. With this masking, the model learns only to generate the correct transcription given the audio input.

3.2.5 Expected Results

Condition	Expected WER	Expected CER
Clean conversational Darija	18–25%	8–15%
Noisy/accented Darija	25–35%	12–20%
MSA (zero-shot, retained)	10–15%	5–8%

Table 3.5: Expected WER/CER ranges after corrected fine-tuning.

These estimates assume a clean training corpus with accurate transcriptions. A WER below 20% would represent state-of-the-art for open-source Darija ASR.

3.3 Frontend Screenshots

The following screenshots illustrate the Sowelni interface across its main interaction states. The warm Moroccan-inspired colour palette, the circular microphone button, and the chat bubble layout are visible throughout.

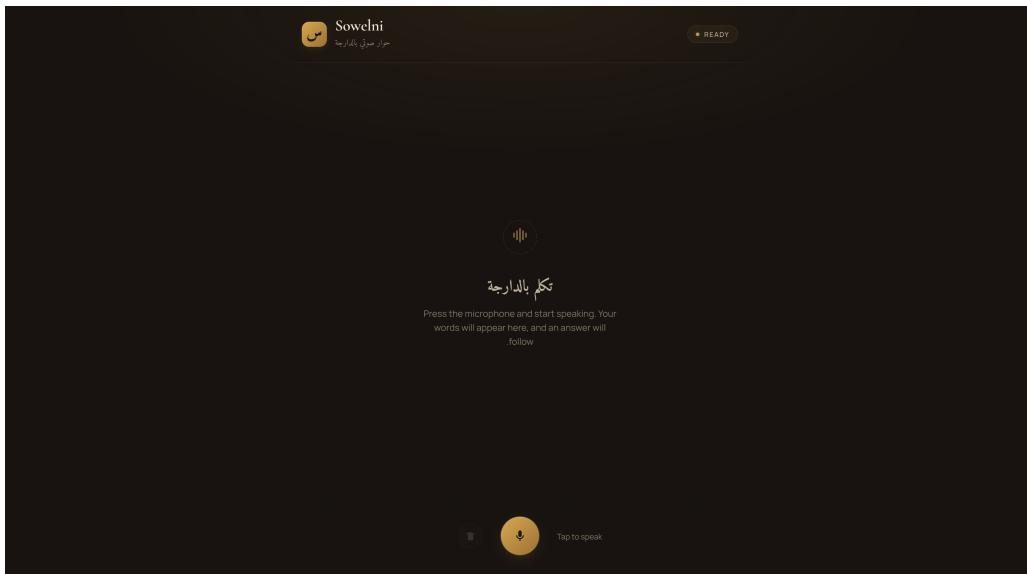


Figure 3.1: Idle state: the application is connected and waiting for the user to press the microphone button.

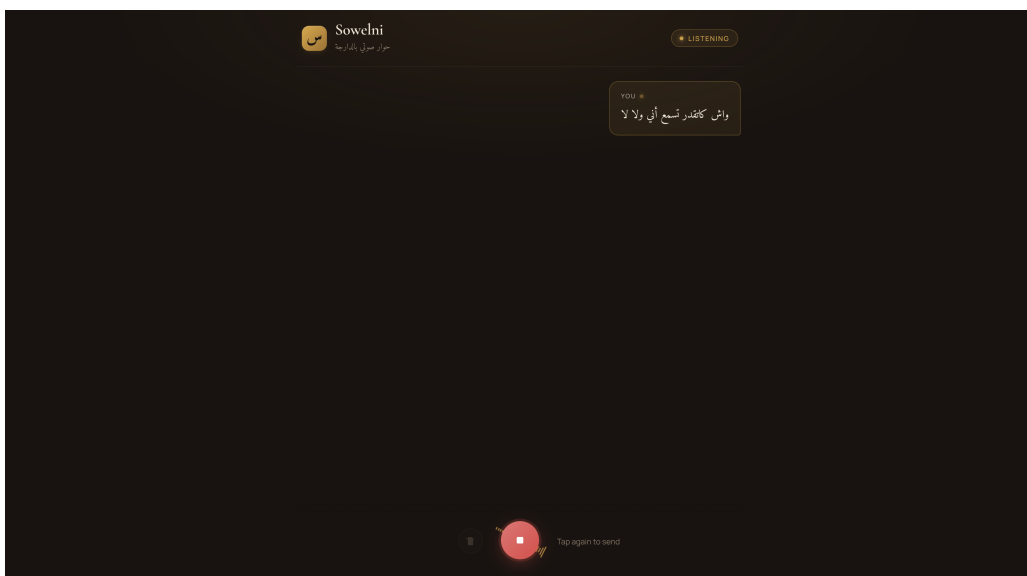


Figure 3.2: Recording state: the microphone button turns red and the 24-bar waveform visualisation animates around it, providing live audio level feedback.

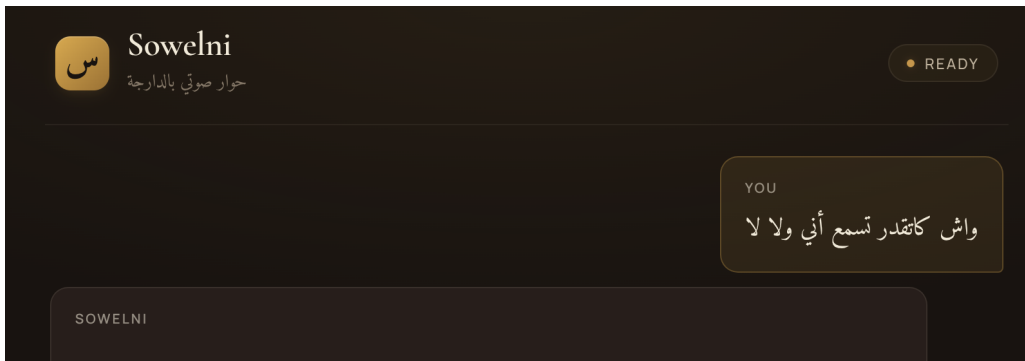


Figure 3.3: Transcription in progress: the Darija speech is transcribed in real time and displayed in the user bubble as each 2-second chunk is processed.

3.4 Deployment and Operations

3.4.1 Startup Sequence

The system starts in four steps. First, the backend loads the ASR model into GPU memory during the FastAPI lifespan handler; the first run downloads the model weights from HuggingFace Hub (approximately 2 GB). Second, the Gemini client is initialised with the API key read from the `.env` file. Third, Uvicorn starts the ASGI server on port 8000. Finally, the Angular development server starts on port 4200 and proxies WebSocket connections to the backend.

3.4.2 Security Considerations

The current configuration uses permissive CORS for local development. For production deployment, origins should be restricted to the actual frontend domain; WebSocket connections must be secured with TLS; rate limiting should be applied to prevent audio flooding from malicious clients; and the `.env` file containing the Gemini API key must never be committed to version control.

This chapter has presented the complete experimental setup, the ASR fine-tuning methodology including a detailed failure analysis and its corrections, a visual walkthrough of the deployed frontend, and the deployment considerations for the Sowelni system.

Conclusion

This report has presented Sowelni, a real-time voice assistant for Moroccan Darija that combines a fine-tuned Qwen3-ASR-0.6B model with an LLM-powered conversational backend and a reactive Angular 19 frontend. The system demonstrates that state-of-the-art ASR for underserved Arabic dialects is achievable with moderate compute resources (a single consumer GPU) and publicly available pre-trained models.

The project encountered and resolved a significant training failure caused by missing label masking and the absence of gradient clipping — a common pitfall in fine-tuning autoregressive speech models that we have documented in detail for the benefit of practitioners. The corrected training methodology provides a complete pipeline from raw audio to structured text. The architecture’s streaming design — audio from browser to server, transcription back per-chunk, and LLM tokens streamed to the UI — ensures low perceived latency despite the computational demands of neural ASR and LLM inference.

Several directions for improvement are planned. Replacing `MediaRecorder` with an `AudioWorkletNode` capturing raw 16 kHz PCM would eliminate the 2-second buffering window and reduce end-to-end latency below 200 ms. Replacing Gemini with a local Qwen3-0.6B model would eliminate the only external dependency, enabling fully offline operation. Integrating Silero VAD on the frontend would enable automatic speech segmentation, removing the need for a manual start/stop button. Adding a TTS component would make Sowelni a true voice-to-voice assistant. Expanding the Darija training data from 40 hours to 200+ hours through semi-supervised methods could significantly reduce WER. Finally, for multi-speaker scenarios, integrating a speaker diarization model would enable per-speaker transcription.

Sowelni is released as open-source software and is designed to be adapted for other low-resource Arabic dialects with minimal modification.

References

- [1] Google. *Angular: The Modern Web Developer’s Platform*. <https://angular.dev/>. 2024.
- [2] Edward Hu, Yelong Shen, Phillip Wallis, et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2022. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685).
- [3] Mozilla Developer Network. *MediaStream Recording API*. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>. 2024.
- [4] Qwen Team. *Qwen3-Omni: A Unified Multimodal Foundation Model*. <https://arxiv.org/abs/2601.21337>. 2025.
- [5] Qwen Team. *Qwen3-ASR: Open-Source Multilingual Automatic Speech Recognition*. <https://huggingface.co/Qwen/Qwen3-ASR-0.6B>. 2026.
- [6] Alec Radford, Jong Wook Kim, Tao Xu, et al. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: [2212.04356](https://arxiv.org/abs/2212.04356).
- [7] Sebastian Ramirez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. <https://fastapi.tiangolo.com/>. 2019.
- [8] James Robert. *Pydub: Manipulate Audio with a Simple and Easy High Level Interface*. <https://github.com/jiaaro/pydub>. 2023.
- [9] Socket.IO Contributors. *Socket.IO: Bidirectional and Low-Latency Communication*. <https://socket.io/>. 2023.
- [10] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [11] W3C. *Web Audio API*. <https://www.w3.org/TR/webaudio/>. 2024.